

# Lectura 1

## Introducción a los Métodos Numéricos

Dr. Luciano Ponzellini Marinelli  
lucianoponzellini@uca.edu.ar

Licenciatura en Ciencias de Datos  
Facultad de Química e Ingeniería  
Rosario, Argentina

Notas de clase  
4 ago 2025





# Organización

DESCRIPCIÓN

PROPUESTA

INTRODUCCIÓN

EJEMPLO

APLICACIONES

# DESCRIPCIÓN DE LA ASIGNATURA

# Objetivos

## Objetivos de la asignatura Métodos Numéricos:

- Adquirir conocimientos y habilidades con base en la resolución de problemas reales usando métodos numéricos.
- Desarrollar la capacidad de aprender e implementar métodos numéricos de forma autónoma.
- Definir un problema numérico determinado e identificar el método numérico adecuado a dicho problema.
- Desarrollar formas de pensamiento lógico-analíticas y experimentales.
- Desarrollar creatividad, pensamiento crítico, y ser capaz de abordar proyectos de investigación a partir de casos de aplicación.

# Objetivos

|                                          |                             |
|------------------------------------------|-----------------------------|
| Carrera: Lic. en Ciencias de Datos       | Curso lectivo 2025          |
| Asignatura: Métodos y Cómputos Numéricos | Ubicación: 2º Año / 2º Sem. |
| Duración: cuatrimestral (16 semanas)     | Horas totales: 80           |
| Turno: Tarde                             | Aula: Laboratorio           |

## Descripción

- Clases teórico-prácticas en aulas-laboratorios (días martes).
- Laboratorio experimental en aulas-laboratorios (días luees).
- Rol activo al discutir la solución a la problemática que se plantea
- Utilización de herramientas computacionales para verificar los resultados obtenidos.
- Actividades prácticas orientadas al trabajo computacional
- Proyecto integrador aplicado (ver propuesta didáctica).

# Laboratorios UCA

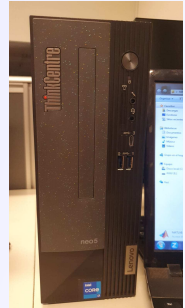


Figura: Laboratorio y aulas con terminales.

# Detalles del cursado

Durante las 16 semanas de cursado se realizarán:

## Sistema de evaluación

- **Diagnóstico** (en máquina) sobre manejo de software de cálculo numérico (Matlab, Octave, Scilab o Python) relacionados con su sintaxis, manejo de funciones, scripts y ploteos.
- **Evaluación 1** (en máquina) sobre errores en los métodos numéricos, punto flotante, raíces y resolución de sistemas de ecuaciones lineales por métodos directos e iterativos.
- **Evaluación 2** (en máquina) sobre resolución de sistemas de ecuaciones no lineales, interpolación, ajuste y optimización.
- **Recuperatorio** de a lo sumo 1 (una) de las dos evaluaciones.
- **Proyecto final integrador aplicado**. En equipos.

## Bibliografía sugerida

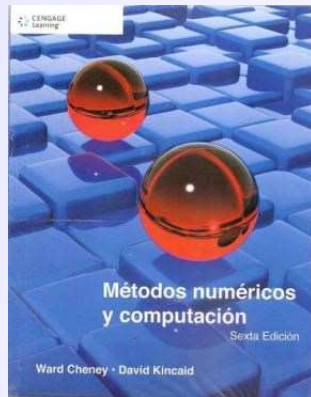
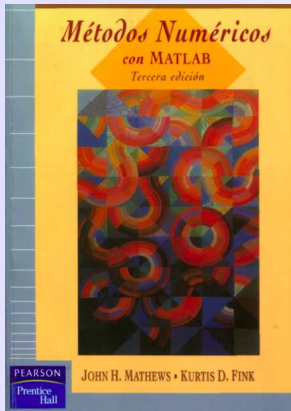


Figura: Libros sugeridos para seguimiento de la asignatura.



## Otras libros complementarios sugeridos

- H. Golub y C.F. Van Loan, “Matrix Computations”, Johns Hopkins Studies in Mathematical Sciences, 1996.
- C.T. Kelley, “Solving Nonlinear Equations with Newton’s Method”, Society for Industrial and Applied Mathematics, Philadelphia, 2003.
- D. Etter, “Solución de problemas de Ingeniería con Matlab”, Prentice Hall, 1999.
- C. Moler, “Numerical Computing with Matlab”, Society for Industrial and Applied Mathematics, Philadelphia, 2004.
- S. Nakamura, “Análisis Numérico y visualización gráfica con Matlab”, Prentice Hall, 1997.
- L.I.N. Trefethen y D. Bau, “Numerical Linear Algebra”, Society for Industrial and Applied Mathematics, Philadelphia, 2022.

# PROPUESTA DIDÁCTICA

# Propuesta didáctica

## Aprendizaje Basado en Problemas (ABP)

- Metodología de proceso **enseñanza-aprendizaje** diferente a la didáctica tradicional centrado en las clases magistrales
- El estudiante es considerado **sujeto activo** y protagonista de su propio proceso de aprendizaje
- El docente **orienta**, facilita y estimula
- **Preguntas** relacionadas con contenidos que puedan aplicarse y traducirse a casos concretos
- Actividades basadas en la **resolución de problemas** vinculados a esas preguntas
- Promueve el **trabajo autónomo**, el compromiso y la cooperación
- Apunta a la elaboración de un **producto final** para ser compartido

# Propuesta didáctica

## Trabajo Integrador Aplicado

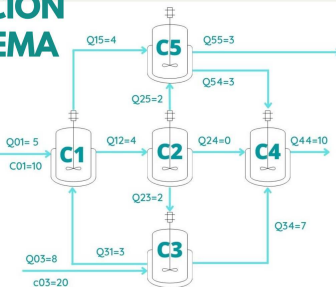
- Consiste en la presentación de un problema de aplicación real a las ingenierías elegido por el o la estudiante que involucre la resolución de un problema numérico real usando uno o varios métodos numéricos vistos en clase o no.
- El proyecto se acompaña con consultas para guiar el trabajo y definir cuando es apto para su defensa oral.
- Una vez es apto a defensa, se realiza un **Reporte técnico** (máximo 10 páginas) y una **Presentación** en diapositivas con proyector en forma presencial en grupos de dos o tres personas.
- La duración de la presentación será de 15 minutos de reloj y posteriormente habrá 5 minutos de preguntas.

# Trabajo integrador aplicado 1

*"Balance de materia en reactores químicos" (2022)*

## PRESENTACIÓN DEL PROBLEMA

Emplee la conservación de la masa para determinar las concentraciones en estado estacionario del siguiente sistema de reactores químicos conectados

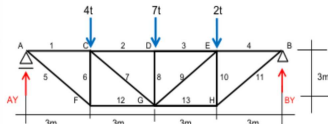


- Problema ubicado en el área de aplicación de la Ingeniería Química.
- Ecuación balance de materia en estado estacionario.
- Método de Descomposición LU en Scilab.

## Trabajo integrador aplicado 2

*“Fuerzas e interacciones causa-efecto asociadas a una armadura estáticamente determinada”*

### Estructura 2



- Armaduras estáticamente determinadas.
- Resolución de SEL (uno de estos “sparse”).
- Métodos directos e iterativos (GMRES).

## Trabajo integrador aplicado 3

*“Análisis de caída de presión en una tubería” (2023)*

### Flujo en tuberías

Al movilizarse el líquido o el gas en tuberías o ductos, el fluido usualmente se fuerza a fluir mediante un ventilador o bomba que atraviesa un tramo del ducto. Se pone particular atención a la fricción, que se relaciona directamente con la caída de presión y las pérdidas de carga en el flujo. Entonces, la **caída de presión** se usa para determinar la **potencia necesaria de bombeo**.



- Cálculo de la caída de presión en una sección de un tubo.
- Resolución de la ecuación de Colebrook para el factor de fricción.
- Método de Newton-Raphson y falsa posición.

## Trabajo integrador aplicado 3

*“Análisis de caída de presión en una tubería” (2023)*

### Flujo en tuberías

Al movilizarse el líquido o el gas en tuberías o ductos, el fluido usualmente se fuerza a fluir mediante un ventilador o bomba que atraviesa un tramo del ducto. Se pone particular atención a la fricción, que se relaciona directamente con la caída de presión y las pérdidas de carga en el flujo. Entonces, la **caída de presión** se usa para determinar la **potencia necesaria de bombeo**.

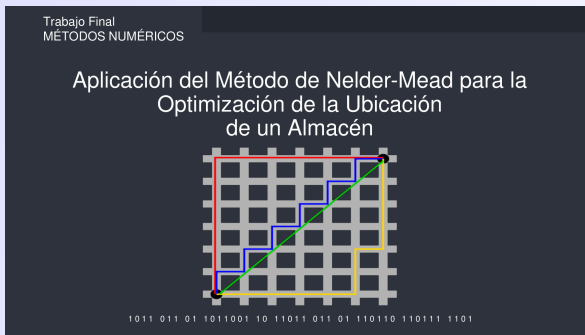


- Cálculo de la caída de presión en una sección de un tubo.
- Resolución de la ecuación de Colebrook para el factor de fricción.
- Método de Newton-Raphson y falsa posición.



# Trabajo integrador aplicado 4

## *“Aplicación del Método de Nelder-Mead para la Optimización de la Ubicación de un Almacén” (2024)*



- Minimización de una función objetivo en un caso de testeo y real.
- Técnica de optimización que utiliza un cuerpo geométrico (simplex).
- Método de Nelder-Mead.

# INTRODUCCIÓN A LOS MÉTODOS NUMÉRICOS

# Introducción a los Métodos Numéricos

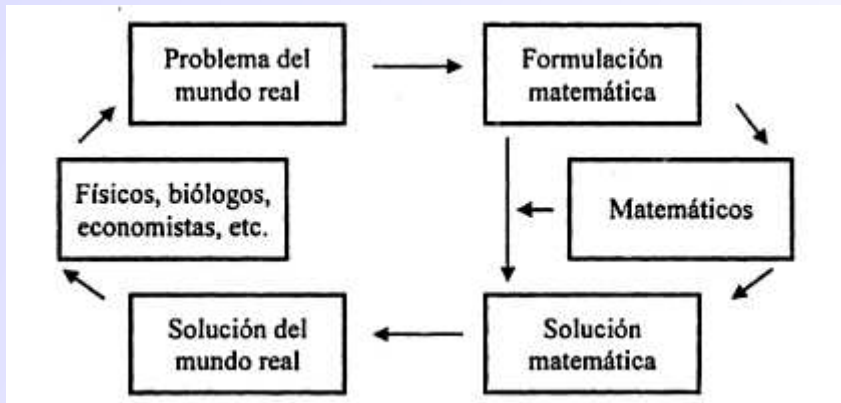


Figura: Representación esquemática en la resolución de problemas

# Introducción a los Métodos Numéricos

- Los *modelos matemáticos* son herramientas para resolver problemas científicos que se presentan en las distintas ciencias, en la industria, en la gestión pública o privada, entre otros
- Usando leyes generales de la *física, química, economía*, etc., se obtienen *ecuaciones* que modelan el problema en estudio
- Para modelar el problema, se hacen *simplificaciones* por lo que el modelo matemático no describe exactamente *la realidad*
- No siempre podemos hallar analíticamente *soluciones exactas* de nuestros problemas, por ende tenemos que recurrir a una *solución aproximada* del problema
- Para construir esas *soluciones aproximadas* necesitamos desarrollar, implementar y validar los *métodos numéricos*

# Introducción a los Métodos Numéricos

Las *aproximaciones numéricas* introducidas al *modelo matemático*, reemplazan el *problema matemático* por un *problema numérico*

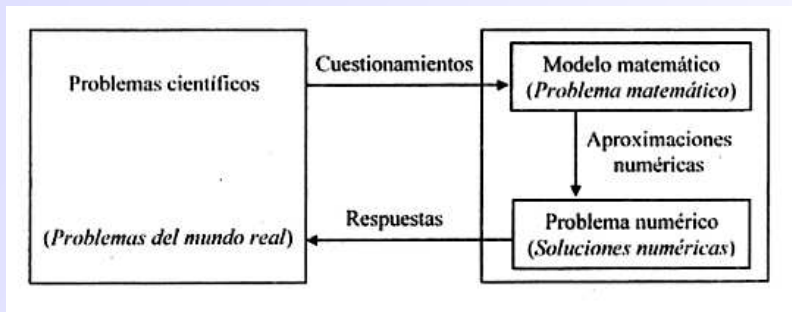


Figura: Representación alternativa

# Introducción a los Métodos Numéricos

- Un problema numérico determinado necesita ser resuelto por un **algoritmo** o **método numérico** apropiado
- Los métodos numéricos son desarrollados teóricamente, implementados computacionalmente y testeados numéricamente
- Permiten simplificar procedimientos y cálculos matemáticos en una computadora
- Su desarrollo ha sido determinado por su uso ingenieril como el **Método de Elementos Finitos** (MEF en castellano o FEM en inglés)
- Su estudio se ha consolidado como ciencia a partir del desarrollo de las computadoras y la Tecnología
- El area de la Matemática que estudia los fundamentos de los métodos numéricos y analizar sus errores se conoce como **Análisis Numérico**

# ¿Qué es el *Análisis Numérico*?

Según Trefethen & Bau en “*Numerical Linear Algebra*” [TrBa1997]:

Respuesta equivocada:

*El Análisis Numérico es el estudio de los errores de redondeo y de la aritmética de punto flotante*

Respuesta “más” acertada:

*El Análisis Numérico es el estudio de algoritmos para problemas de la matemática continua*

*Pero la mayoría de los problemas de la matemática continua no pueden ser resueltos por algoritmos finitos*

- Sistema de Ecuaciones Lineales (SEL)
- Raíces de ecuaciones
- Sistemas de Ecuaciones No Lineales (SENL)
- Interpolación y aproximación de funciones
- Derivación numérica
- Cuadraturas o integraciones numéricas
- Optimización numérica. Programación lineal
- Ecuaciones Diferenciales Ordinarias (EDO)
- Ecuaciones Diferenciales Parciales (EDP)



## ¿Qué es un *problema* y un *método numérico*?

**Problema numérico:** Descripción clara sin ambigüedades que relaciona datos de entrada (data) con datos de salida (soluciones). Estos datos consisten en un número finito de cantidades reales.

**Ejemplo:** Aproximar  $\sqrt{2}$  con cierta tolerancia

**Método numérico:** Descripción completa de un número finito de operaciones bien definidas para un problema numérico dado, a través de las cuales cada vector de datos de entrada es transformado en un vector de datos de salida.

**Ejemplo:** Esquema iterativo o desarrollo de Taylor que aproxime  $\sqrt{2}$

# Características deseables en un algoritmo

- **Finito:** Debe terminar en un número finito de pasos
- **Definido:** Las acciones deben definirse sin ambigüedad
- **Entrada (input):** Puede tener una o varias entradas
- **Salida (output):** Debe tener una o varias salidas
- **Eficiente:** Las operaciones deben realizarse en un determinado lapso de tiempo tolerable
- **Exacto y preciso** Las salidas del algoritmo deben aproximarse a la solución del problema y entre sí
- **Estable:** Pequeños cambios iniciales en el *input* deben generar pequeños cambios en el *output*

## ¿Cómo implementamos nuestros algoritmos?



- **Fortran** (Formula Translating System)

Lenguaje de programación de alto nivel especialmente adaptado al cálculo numérico y la computación científica. Necesita un compilador para traducir al código máquina. Desarrollado en IBM en 1957 por el matemático **J. Backus** de uso vigente y en desarrollo (FORTRAN 66/77, Fortran 90/95/2003/2008/2018).

[www.fortran.com/](http://www.fortran.com/)

- Otros: **C**, **C++**, **Java**.

## ¿Cómo implementamos nuestros algoritmos?



- **MATLAB** (MATrix LABoratory)

Software privado de MathWorks creado por el matemático y programador **C. Moler** en 1984. Funciona como un intérprete permitiendo ejecutar órdenes en modo interactivo. Tiene un lenguaje propio de programación. Existen una versión de prueba por 30 días.

[www.mathworks.com/products/matlab/](http://www.mathworks.com/products/matlab/)



- **OCTAVE** (GNU Octave)

Software libre parte del **Proyecto GNU** para realizar cálculos numéricos (equivalente versión libre de MATLAB). A diferencia de Mathematica o Maple (sistemas de álgebra computacional) es orientado al análisis y cálculo numérico. Creado en 1988 para la ingeniería química y desde 1992 el ingeniero químico **J.W. Eaton** sigue su desarrollo.

[www.gnu.org/software/octave/](http://www.gnu.org/software/octave/)

# ¿Cómo implementamos nuestros algoritmos?



- **Scilab**

Software para análisis numérico con lenguaje de programación para cálculo científico compatible con licencia GNU. Desarrollado en los 80's en el INRIA (Francia) por los programadores **F. Delebecque** & **S. Steer** y desde julio de 2012 desarrollado por Scilab Enterprises.

[www.scilab.org/](http://www.scilab.org/)

# ¿Cómo implementamos nuestros algoritmos?



- **Python**

Lenguaje de programación de alto nivel cuya filosofía hace hincapié en la legibilidad del código. Es multiparadigma (orientado a objetos). Es un lenguaje interpretado, dinámico y multiplataforma. Fue creado a fines de los 80's por **Guido van Rossum** en Stichting Mathematisch Centrum, Países Bajos. A partir del 2001 Python Software Foundation (PSF), organización sin ánimo de lucro, pasa a dirigir el proyecto.

<https://www.python.org/>



- **Julia**

Lenguaje de programación multiplataforma y multiparadigma ("programación orientada a objetos") de alto nivel y alto desempeño para computación genérica, técnica y científica. Libre y de código abierto. Desarrollado desde 2009 y lanzado en 2012 por **J. Bezanson, S. Karpinski, V.B. Shah & A. Edelman**.

<https://julialang.org/>

## EJEMPLO

## Ejemplo de un algoritmo

Problema numérico: Calcular  $\sqrt{2}$  con una determinada *tolerancia*.

Solució numérica: Usaremos el siguiente *esquema iterativo*

$$\begin{aligned} x_0 &= 1, \\ x_{n+1} &= \frac{1}{2} \left( x_n + \frac{2}{x_n} \right), \quad n \geq 0. \end{aligned}$$

### ALGORITMO:

**Entrada:**  $x_0$  (aproximación inicial),  $tol$  (tolerancia)

**Paso 1:**  $x_1 \leftarrow \frac{1}{2} \left( x_0 + \frac{2}{x_0} \right)$

**Paso 2:** Si  $|x_1 - x_0| \leq tol$  entonces TERMINAR (STOP)

En caso contrario, hacer  $x_0 \leftarrow x_1$  y repetir **Paso 1**

**Paso 3:** Asignar la salida a  $x_1$

**Paso 4:** TERMINAR (STOP)

**Salida:**  $x_1$  aproximación de  $\sqrt{2}$



# Ejemplo de un algoritmo en MATLAB

```
function x1 = rcuad2(x0,tol)
% rcuad.m aproxima la raiz cuadrada de 2.
% INPUT: x0: aproximacion inicial de la raiz de 2.
% tol : (optional) tolerancia admitida (valor por defecto: 5e-9).
% OUTPUTt: x1 : valor aproximado con dicha tolerancia.
clc % limpio salida MATLAB
if nargin < 2, tol = 5e-9; % tolerancia por defecto
end
fprintf('Valor inicial aproximado: %g  n n n raiz error n',x0)
x1 = .5*(x0+2/x0); % inicio aproximacion
err = abs(x1-x0); % calculo error entre aproximaciones
k=1; % contador
fprintf(' %3d %12.15f %12.15f n',k,x1,err) % imprimo valor inicial
...
```

# Ejemplo de un algoritmo en MATLAB

```
...
while err>tol % verifico convergencia
    x0 = x1;
    x1 = .5*(x0+2/x0); % actualizo termino aproximado
    err = abs(x1-x0);
    k = k + 1;
    fprintf(' %3d %12.15f %12.15f n',k,x1,err) % imprimo valores
    aproximados
end
fprintf(' n Valor final aproximado: %12.15f n',x1) % output
fprintf('Valor exacto MATLAB : %12.15f n',sqrt(2)) % exacto
fprintf('Error absoluto : %12.15f n',abs(x1-sqrt(2))) % error final
end
```

# Resultado de un algoritmo en MATLAB

```
>> r = rcuad2(1,1e-5)
```

Valor inicial aproximado: 1

n raiz

error

1 1.5000000000000000 0.5000000000000000

2 1.4166666666666667 0.0833333333333333

3 1.414215686274510 0.002450980392157

4 1.414213562374690 0.000002123899820

Valor final aproximado: 1.414213562374690

Valor exacto MATLAB : 1.414213562373095

Error absoluto : 0.000000000001595

$r = 1.414213562374690$

¿Qué pasa si modifico la *aproximación inicial* a  $x_0 = 1,5$ ?

```
>> r = rcuad2(1.5,1e-5)
```

Valor inicial aproximado: 1.5

n raiz

error

```
1 1.416666666666667 0.0833333333333333
```

```
2 1.414215686274510 0.002450980392157
```

```
3 1.414213562374690 0.000002123899820
```

Una iteración menos!

Valor final aproximado: 1.414213562374690

Valor exacto MATLAB : 1.414213562373095

Error absoluto : 0.000000000001595

```
r = 1.414213562374690
```

Misma aproximación anterior

Tomando inicialmente un valor más cercano a  $\sqrt{2}$  el algoritmo también converge y lo hace en menos iteraciones.

¿Qué pasa si modifico la *aproximación inicial* a  $x_0 = 5$ ?

```
>> r = rcuad2(5,1e-5)
```

Valor inicial aproximado: 5

n raiz

error

1 2.7000000000000000 2.3000000000000000

2 1.720370370370370 0.979629629629630

3 1.441455368177650 0.278915002192720

4 1.414470981367771 0.026984386809879

5 1.414213585796884 0.000257395570888

6 1.414213562373095 0.000000023423788

Dos iteraciones más!

Valor final aproximado: 1.414213562373095

Valor exacto MATLAB : 1.414213562373095

Error absoluto : 0.0000000000000000

$r = 1.414213562373095$

Mejor aproximación!

¿Qué pasa si aumento la *tolerancia* a  $tol = 5 \times 10^{-9}$ ?

```
>> r = rcuad2(1)      (por defecto tol=5e-9)
```

Valor inicial aproximado: 1

| n raiz               | error              |
|----------------------|--------------------|
| 1 1.5000000000000000 | 0.5000000000000000 |
| 2 1.4166666666666667 | 0.0833333333333333 |
| 3 1.414215686274510  | 0.002450980392157  |
| 4 1.414213562374690  | 0.000002123899820  |
| 5 1.414213562373095  | 0.000000000001595  |

Una iteración más!

Valor final aproximado: 1.414213562373095

Valor exacto MATLAB : 1.414213562373095

Error absoluto : 0.0000000000000000

$r = 1.414213562373095$

¿Qué pasa si aumento la *tolerancia* a  $tol = 1 \times 10^{-12}$ ?

```
>> r = rcuad2(1,1e-12)
```

Valor inicial aproximado: 1

| n raiz               | error              |
|----------------------|--------------------|
| 1 1.5000000000000000 | 0.5000000000000000 |
| 2 1.4166666666666667 | 0.0833333333333333 |
| 3 1.414215686274510  | 0.002450980392157  |
| 4 1.414213562374690  | 0.000002123899820  |
| 5 1.414213562373095  | 0.000000000001595  |
| 6 1.414213562373095  | 0.0000000000000000 |

Dos iteraciones más!

Valor final aproximado: 1.414213562373095

Valor exacto MATLAB : 1.414213562373095

Error absoluto : 0.0000000000000000

$r = 1.414213562373095$

## Otra forma de ejecutar algoritmos: *scripts*

- Creamos un nuevo archivo que denominaremos “script” (en MATLAB y Octave de extensión .m, en Scilab .sce)
- En ese archivo describimos secuencialmente las instrucciones que queremos se realicen al ejecutarse este script

**t\_rcuad2.m**

```
% script para ejecutar rcuad2.m  
r0 = 1; % defino la aprox inicial  
tol = 1e-5; % defino la tol inicial  
r = rcuad2(r0,tol)
```

- Ejecutamos este script en la *Ventana de comandos*

```
>> t_rcuad2
```

Esto también puede hacerse desde un acceso directo del programa.



# APLICACIONES EN LAS INGENIERÍAS Y LA MATEMÁTICA APLICADA

### Congresos:

- X Congreso de Matemática Aplicada, Computacional e Industrial (MACI 2025) 12 al 15 de Mayo, Córdoba.
- XLI Congreso Argentino de Mecánica Computacional (MECOM 2025) 11 al 14 de Noviembre, Buenos Aires.

### Asociaciones:

- Asociación Argentina de Mecánica Computacional (MECOM)
- Society for Industrial and Applications of Mathematics (SIAM)
- Asociación Argentina de Matemática Aplicada, Computacional e Industrial (ASAMACI)

### Revistas internacionales:

- Computers and Mathematics with Applications
- International Journal for Numerical Methods in Engineering
- Mathematics and Computers in Simulation

# Bibliografía

- **[Alav2006]** J. Alavez Ramírez, *“Métodos Numéricos I”*, Universidad Juárez Autónoma de Tabasco, México, 2006.
- **[Ette1998]** D.M. Etter, *Solución de problemas de ingeniería con MATLAB*, Pearson Prentice Hall, Madrid, España, 1998.
- **[MaFi2000]** J.M. Mathews - K.D. Fink, *Métodos Numéricos con MATLAB*, 3era Ed., Pearson Prentice Hall, Madrid, 2000.
- **[Mole2015]** C.B. Moler, *“Numerical computing with MATLAB”*, SIAM, Philadelphia, 2004.
- **[SiSo2010]** J.W. Signorelli & J. Sorribas, *“Notas de Clases de Informática Aplicada y Métodos Computacionales”*, FCEIA, UNR, 2010.
- **[TrBa1997]** L.I.N. Trefethen & D. Bau, *“Numerical Linear Algebra”*, SIAM, Philadelphia, 1997.